

吴恩达 NLP 课程 2

目录

吴恩达 NLP 课程 2.....	1
1. N-gram 模型.....	2
1.1 简介	2
1.2 什么是 N-Gram?	3
1.3 文本序列概率的计算	6
1.4 句子的开头和结束部分	8
1.5 N-gram 语言模型	13
1.6 语言模型评估	16
1.7 词汇量匮乏时	19
1.8 平滑处理	21
2. 词嵌入.....	24
2.1 大纲介绍	24
2.2 独热向量	24
2.3 词嵌入	24
2.4 词嵌入方法模型介绍	26
2.5 连续词袋模型 (CBOW)	27
2.6 清理语料库、令牌化 (标记化过程)	29
2.7 将单词转化为向量	33

2.8 CBOW 的架构和原理推导.....	34
2.9 算法推导	39
2.9.1 代价函数.....	39
2.9.2 正向传播	41
2.9.3 反向传播和梯度下降	42
2.10 提取词嵌入向量	43
2.11 词嵌入的评估.....	46
2.11.1 内部评估.....	46
2.11.2 外部评估.....	46

1. N-gram 模型

1.1 简介

首先，您将把原始文本语料库转换为一个语言模型，该模型通过使用句子的前一个单词返回下一个单词的概率。

接下来，您将调整您的语言模型，以处理在训练期间未看到的单词。这些词是用 UNK 来称呼的。平滑是另一种技术，您可以使用它来处理以前未看到的输入。这种方法仍然可以成功地估计看不见的单词和句子的概率。平滑本质上假装每个单词和短语至少出现在训练语料库中一次。这有助于计算概率，即使是对于不寻常的单词和序列。

最后，我将向您展示如何使用混淆度量来选择最佳的语言模型，这是一个新的工具。

1.2 什么是 N-Gram?

一个 N-gram 就是一个单词序列，而且是具有单词顺序的序列。处理语料库时，标点符号被视为单词。但是所有其他特殊字符（例如代码）将被删除。

N-gram

An N-gram is a sequence of N words

Corpus: I am happy because I am learning

Unigrams: { I , am , happy , because , learning }

Bigrams: { I am , am happy , happy because ... }

✗ I happy

Trigrams: { I am happy am happy because, ... }

上面三个集合分别是，单元组单词序列，双元组单词序列和三元组单词序列。

Sequence notation

Corpus: This is great ... teacher drinks tea. $m = 500$
 $w_1 \ w_2 \ w_3 \qquad w_{498} \ w_{499} \ w_{500}$

$$w_1^m = w_1 \ w_2 \ ... \ w_m$$

$$w_1^3 = w_1 \ w_2 \ w_3$$

$$w_{m-2}^m = w_{m-2} \ w_{m-1} \ w_m$$

下角标指的是单词开始计数的索引，上角标指的是结束计数的索引。最后一个指的是最后三个单词构成的三元组序列。

Unigram probability

Corpus: I am happy because I am learning

Size of corpus $m = 7$

$$P(I) = \frac{2}{7}$$

$$P(happy) = \frac{1}{7}$$

Probability of unigram: $P(w) = \frac{C(w)}{m}$

Bigram probability

Corpus: I am happy because I am learning

$$P(am|I) = \frac{C(I\ am)}{C(I)} = \frac{2}{2} = 1$$

$$P(happy|I) = \frac{C(I\ happy)}{C(I)} = \frac{0}{2} = 0 \quad \times \text{ I happy}$$

$$P(learning|am) = \frac{C(am\ learning)}{C(am)} = \frac{1}{2}$$

Probability of a bigram: $P(y|x) = \frac{C(x\ y)}{\sum_w C(x\ w)} = \frac{C(x\ y)}{C(x)}$

can be estimated as the counts

Trigram Probability

Corpus: I am happy because I am learning

$$P(\text{happy} | \text{I am}) = \frac{C(\text{I am happy})}{C(\text{I am})} = \frac{1}{2}$$

Probability of a trigram: $P(w_3 | w_1^2) = \frac{C(w_1^2 w_3)}{C(w_1^2)}$

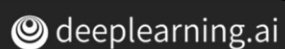
$$C(w_1^2 w_3) = C(w_1 w_2 w_3) = C(w_1^3)$$

N-gram probability

Probability of N-gram: $P(w_N | w_1^{N-1}) = \frac{C(w_1^{N-1} w_N)}{C(w_1^{N-1})}$

$$C(w_1^{N-1} w_N) = C(w_1^N)$$

You've also calculated their probability



from a corpus by counting their

1.3 文本序列概率的计算

Probability of a sequence

- Given a sentence, what is its probability?

$$P(\textit{the teacher drinks tea}) = ?$$

- Conditional probability and chain rule reminder

$$P(B|A) = \frac{P(A, B)}{P(A)} \implies P(A, B) = P(A)P(B|A)$$

$$P(A, B, C, D) = P(A)P(B|A)P(C|A, B)P(D|A, B, C)$$

Probability of a sequence

$$P(\textit{the teacher drinks tea}) =$$

$$P(\textit{the})P(\textit{teacher}|\textit{the})P(\textit{drinks}|\textit{the teacher}) \\ P(\textit{tea}|\textit{the teacher drinks})$$

您刚刚看到的只是理想情况，训练语料库中确实足够的数据来计算所有这些概率。但是，这种直接的序列概率方法有其局限性。由于自然语言是高度可变的，因此您的整个查询语句甚至句子中较长的部分，都不太可能出现在训练语料库中，因此其概率往往会出现为 0 的情况，影响条件概率的计算。

Sentence not in corpus

- Problem: Corpus almost never contains the exact sentence we're interested in or even its longer subsequences!

Input: *the teacher drinks tea*

$$P(\text{the teacher drinks tea}) = P(\text{the})P(\text{teacher}|\text{the})P(\text{drinks}|\text{the teacher})P(\text{tea}|\text{the teacher drinks})$$

$$P(\text{tea}|\text{the teacher drinks}) = \frac{C(\text{the teacher drinks tea})}{C(\text{the teacher drinks})} \leftarrow \begin{array}{l} \text{Both} \\ \text{likely 0} \end{array}$$

因此，我们采取了近似的想法。和 Markov 转移概率的想法类似，每一步的转移只和上一次的状态有关。

分享

Approximation of sequence probability

the teacher drinks tea

$$P(\text{tea}|\text{the teacher drinks}) \approx P(\text{tea}|\text{drinks})$$

$$\begin{array}{l} P(\text{teacher}|\text{the}) \\ P(\text{drinks}|\text{teacher}) \\ P(\text{tea}|\text{drinks}) \end{array}$$

$$\begin{array}{l} P(\text{the teacher drinks tea}) = \\ P(\text{the})P(\text{teacher}|\text{the})P(\text{drinks}|\text{the teacher})P(\text{tea}|\text{the teacher drinks}) \\ \downarrow \\ P(\text{the})P(\text{teacher}|\text{the})P(\text{drinks}|\text{teacher})P(\text{tea}|\text{drinks}) \end{array}$$

predecessors, in other words, the product of the probabilities of Bigrams.

您只需应用马尔可夫假设，而不是查找所有单词，您只是看了前一个词。您还可以考虑前面的两个单词，或者任意 n 个先前的单词。马尔可夫假设表明每个单词的概率取决于仅限于长度为 n 的有限历史。它会忽略所有在指定长度 N 之前出现的单词。因此对于 Bigrams，您可以近似条件，给定历史 W_1 至 W_{n-1} 时单词 W_n 的概率为：给定单词 W_{n-1} 时单词 W_n 的条件概率。这意味着您只需要查看每个单词前面的单词即可。使用 N -gram 进行计算时，您只会看到 $n-1$ 个先前的单词 连锁规则中的条件概率。估计句子概率的按地公式现在是，单词及其直接前面 $N-1$ 个单词的条件概率。

Approximation of sequence probability

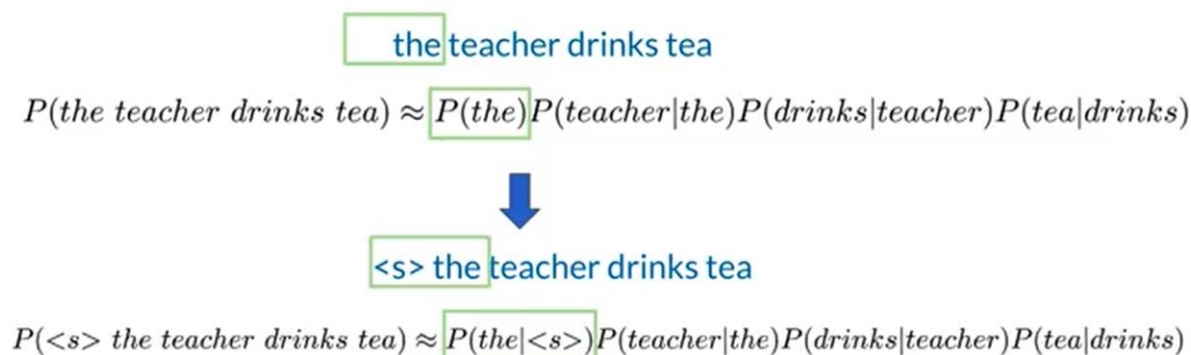
- Markov assumption: only last N words matter
- Bigram $P(w_n|w_1^{n-1}) \approx P(w_n|w_{n-1})$
- N-gram $P(w_n|w_1^{n-1}) \approx P(w_n|w_{n-N+1}^{n-1})$
- Entire sentence modeled with bigram $P(w_1^n) \approx \prod_{i=1}^n P(w_i|w_{i-1})$
 $P(w_1^n) \approx P(w_1)P(w_2|w_1)\dots P(w_n|w_{n-1})$

1.4 句子的开头和结束部分

上节课讲述了句子的概率计算，但是对于开头单词的概率计算，往往需要我们预估，本节课我们会添加 S 这样的符号标记，来对开头或结尾的条件概率进行一个相对准确的概率估计。

- 开头的预估

Start of sentence token <s>



如果是 N-gram 模型，需要在开头部分添加 N-1 个 S 的标记。

Start of sentence token <s> for N-grams

- Trigram:

$$P(\text{the teacher drinks tea}) \approx$$

$$P(\text{the})P(\text{teacher}|\text{the})P(\text{drinks}|\text{the teacher})P(\text{tea}|\text{teacher drinks})$$

the teacher drinks tea => <s> <s> the teacher drinks tea

$$P(w_1^n) \approx P(w_1|<s> <s>)P(w_2|<s> w_1) \dots P(w_n|w_{n-2} w_{n-1})$$

- N-gram model: add N-1 start tokens <s>

brackets S at the beginning

- 结尾的预估

当 x 是最后一个单词的时候，会出现数学逻辑上的错误，如下图，对 C (drinks, w) 进行求和，得到的数目并不是 C (drinks)。

End of sentence token </s> - motivation

$$P(y|x) = \frac{C(x y)}{\sum_w C(x w)} = \frac{C(x y)}{C(x)}$$

Corpus:

<s> Lyn drinks chocolate

<s> John drinks

$$\sum_w C(\text{drinks } w) = 1$$

$$C(\text{drinks}) = 2$$

you need to add an end

of sentence token.

举例计算一个 2-gram 模型的例子：

End of sentence token </s> - motivation

Corpus	Sentences of length 2:	
<s> yes no	<s> yes yes	$P(< s > \text{ yes yes}) =$
<s> yes yes	<s> yes no	$P(\text{yes} \mid < s >) \times P(\text{yes} \mid \text{yes}) =$
<s> no no	<s> no no	$\frac{C(< s > \text{ yes})}{\sum_w C(< s > w)} \times \frac{C(\text{yes yes})}{\sum_w C(\text{yes } w)} =$
<s> no yes	<s> no yes	$\frac{2}{3} \times \frac{1}{2} = \frac{1}{3}$

the sentence yes yes
 deeplearning.ai estimated from your corpus.

慢慢学着计算下面的概率

分享

End of sentence token </s> - motivation

Corpus	Sentences of length 2:	
<s> yes no	<s> yes yes	$P(< s > \text{ yes yes}) = \frac{1}{3}$
<s> yes yes	<s> yes no	$P(< s > \text{ yes no}) = \frac{1}{3}$
<s> no no	<s> no no	$P(< s > \text{ no no}) = \frac{1}{3}$
<s> no yes	<s> no yes	$P(< s > \text{ no yes}) = 0$
		$\sum_{2 \text{ word}} P(\dots) = 1$

the sum equals to one exactly
 deeplearning.ai what you were aiming for.

加上了结尾的 S 记号之后，上面出现的数学逻辑错误也被修正了。而且固定长度的单词模型概率和为 1。如下图：

End of sentence token </s> - solution

- Bigram

<s> the teacher drinks tea ==> <s> the teacher drinks tea </s>

$$P(the|<s>)P(teacher|the)P(drinks|teacher)P(tea|drinks)P(</s>|tea)$$

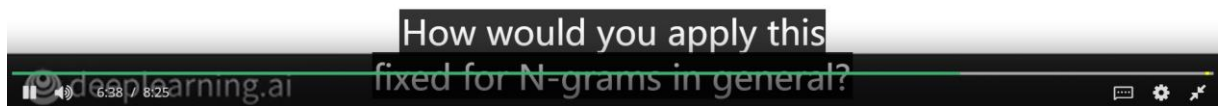
Corpus:

<s> Lyn drinks chocolate </s>

<s> John drinks </s>

$$\sum_w C(drinks\ w) = 2$$

$$C(drinks) = 2$$



末尾只需要加一个 </s> 就可以了。

End of sentence token </s> for N-grams

- N-gram ==> just one </s>

E.g. Trigram:

the teacher drinks tea ==> <s> <s> the teacher drinks tea </s>



计算一下下面的式子，左下角的概率应该错了，正确答案是 1. (1/1)

Example - bigram

Corpus

<s> Lyn drinks chocolate </s>

<s> John drinks tea </s>

<s> Lyn eats chocolate </s>

$$P(\text{sentence}) = \frac{2}{3} * \frac{1}{2} * \frac{1}{2} * \frac{2}{2} = \frac{1}{6}$$

$$P(\text{John}|\text{<s>}) = \frac{1}{3}$$

$$P(\text{</s>}|\text{tea}) = \frac{1}{1}$$

$$P(\text{chocolate}|\text{eats}) = \frac{1}{2}$$

$$P(\text{Lyn}|\text{<s>}) = ? = \frac{2}{3}$$

 deeplearning.ai

chocolate backslash S,
which is two over two.

本节测验

- 你选哪个?

Given these conditional probabilities:

- $P(\text{Mary})=0.1$
- $P(\text{likes})=0.2$
- $P(\text{cats})=0.3$
- $P(\text{Mary}|\text{<s>})=0.2$
- $P(\text{</s>}|\text{cats})=0.6$
- $P(\text{likes}|\text{Mary})=0.3$
- $P(\text{cats}|\text{likes})=0.1$

Approximate the probability of the following sentence with bigrams: “<s> Mary likes cats </s>”

$P(\text{<s> Mary likes cats </s>}) = 0$

$P(\text{<s> Mary likes cats </s>}) = 0.0036$

$P(\text{<s> Mary likes cats </s>}) = 0.003$

$P(\text{<s> Mary likes cats </s>}) = 1$

1.5 N-gram 语言模型

Outline

- Count matrix
- Probability matrix
- Language model
- Log probability to avoid underflow
- Generative language model

the n-gram language model to

deeplearning.ai

- 计数矩阵

Count matrix

$$P(w_n | w_{n-N+1}^{n-1}) = \frac{C(w_{n-N+1}^{n-1}, w_n)}{C(w_{n-N+1}^{n-1})}$$

- Rows: unique corpus (N-1)-grams
- Columns: unique corpus words

- Bigram count matrix

Corpus: <s>I study I learn</s>

"study I" bigram

	<s>	</s>	I	study	learn
<s>	0	0	1	0	0
</s>	0	0	0	0	0
I	0	0	0	1	1
study	0	0	1	0	0
learn	0	1	0	0	0

you increase the value in
the count matrix by one.

deeplearning.ai

双元组模型，行为双元组的前 N-1 个单词，列为双元组的最后一个单词。

- 概率矩阵

Probability matrix

- Divide each cell by its row sum

Corpus: <s>I study I learn</s>

Count matrix (bigram)

	<s>	</s>	I	study	learn	sum
<s>	0	0	1	0	0	1
</s>	0	0	0	0	0	0
I	0	0	0	1	1	2
study	0	0	1	0	0	1
learn	0	1	0	0	0	1

$$P(w_n | w_{n-N+1}^{n-1}) = \frac{C(w_{n-N+1}^{n-1}, w_n)}{C(w_{n-N+1}^{n-1})}$$

$$\text{sum}(\text{row}) = \sum_{w \in V} C(w_{n-N+1}^{n-1}, w) = C(w_{n-N+1}^{n-1})$$

Probability matrix

	<s>	</s>	I	study	learn
<s>	0	0	1	0	0
</s>	0	0	0	0	0
I	0	0	0	0.5	0.5
study	0	0	1	0	0
learn	0	1	0	0	0

The prefix I is followed

by study or by learn.

- 语言模型

Language model

- probability matrix => language model
 - Sentence probability
 - Next word prediction

	<s>	</s>	I	study	learn
<s>	0	0	1	0	0
</s>	0	0	0	0	0
I	0	0	0	0.5	0.5
study	0	0	1	0	0
learn	0	1	0	0	0

Sentence probability:

<s> I learn </s>

$$P(\text{sentence}) =$$

$$P(I | <s>) P(\text{learn} | I) P(</s> | \text{learn}) =$$

$$1 \times 0.5 \times 1 =$$

$$0.5$$

the sentence I learn
next in your corpus.

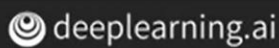
- 对数概率

本截图针对的是 2-gram 模型的计算公式

Log probability

$$P(w_1^n) \approx \prod_{i=1}^n P(w_i | w_{i-1})$$

- All probabilities in calculation ≤ 1 and multiplying them brings risk of underflow
- Logarithm properties reminder $\log(a * b) = \log(a) + \log(b)$



You may recall the
mathematical trick

- 生成语言模型

分享

按 Esc 即可退出全屏模式

Generative Language model

Corpus:

<s> Lyn drinks chocolate </s>

<s> John drinks tea </s>

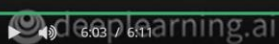
<s> Lyn eats chocolate </s>

1. (<s>, Lyn) or (<s>, John)?
2. (Lyn,eats) or (Lyn,drinks)?
3. (drinks,tea) or (drinks,chocolate)?
4. (tea,</s>) - always

Algorithm:

1. Choose sentence start
2. Choose next bigram starting with previous word
3. Continue until </s> is picked

Now you know how to calculate



n-gram probabilities

当堂测验:

Given the logarithm of these conditional probabilities:

- $\log(P(\text{Mary} | \text{<s>})) = -2$

- $\log(P(</s>|cats))=-1$
- $\log(P(likes|Mary))=-10$
- $\log(P(cats|likes))=-100$

Approximate the log probability of the following sentence with bigrams : “<s> Mary likes cats </s>”

$\log(P(<s> \text{ Mary likes cats } </s>)) = -113$

$\log(P(<s> \text{ Mary likes cats } </s>)) = 2000$

$\log(P(<s> \text{ Mary likes cats } </s>)) = 113$

$\log(P(<s> \text{ Mary likes cats } </s>)) = -112$

1.6 语言模型评估

- 先对数据集进行划分：训练 / 验证（调参） / 测试
- 度量标准：困惑度。复杂度的度量

困惑度得分越小，句子听起来对人耳自然的可能性越大。

Perplexity

E.g. $m=100$

$$P(W) = 0.9 \Rightarrow PP(W) = 0.9^{-\frac{1}{100}} = 1.00105416$$

$$P(W) = 10^{-250} \Rightarrow PP(W) = (10^{-250})^{-\frac{1}{100}} \approx 316$$

- Smaller perplexity = better model

between 60 to 20 sometimes even lower for

Perplexity for bigram models

$$PP(W) = \sqrt[m]{\prod_{i=1}^m \prod_{j=1}^{|s_i|} \frac{1}{P(w_j^{(i)} | w_{j-1}^{(i)})}}$$

$w_j^{(i)} \rightarrow j\text{-th word in } i\text{-th sentence}$

- concatenate all sentences in W

$$PP(W) = \sqrt[m]{\prod_{i=1}^m \frac{1}{P(w_i | w_{i-1})}}$$

$w_i \rightarrow i\text{-th word in test set}$

bigrams in the entire sets.

deeplearning.ai

分享

按 Esc 即可退出全屏模式

Log perplexity

$$PP(W) = \sqrt[m]{\prod_{i=1}^m \frac{1}{P(w_i | w_{i-1})}}$$



$$\log PP(W) = -\frac{1}{m} \sum_{i=1}^m \log_2(P(w_i | w_{i-1}))$$

This is easier to compute.

deeplearning.ai

在一个困惑度介于 20 到 60 之间的好模型中，对数困惑度将在 4.3 到 5.9 之间。

一个例子，说明三元组更贴近人类。

Examples

Training 38 million words, test 1.5 million words, WSJ corpus
Perplexity Unigram: 962 Bigram: 170 Trigram: 109

Unigram

Months the my and issue of year foreign new exchange's september were recession ex-
change new endorsed a acquire to six executives

Bigram

Last December through the way to preserve the Hudson corporation N. B. E. C. Taylor
would seem to complete the major central planners one point five percent of U. S. E. has
already old M. X. corporation of living on information such as more frequently fishing to
keep her

Trigram

They also point to ninety nine point six billion dollars from two hundred four oh six three
percent of the rates of interest stores as Mexico and Brazil on market conditions

[Figure from *Speech and Language Processing* by Dan Jurafsky et. al]

Using a trigram, you can see the language

 deeplearning.ai it produces is pretty close to reasonable.

当堂测验:

Given the logarithm of these conditional probabilities:

- $\log(P(\text{Mary}|\langle s \rangle)) = -2$
- $\log(P(\langle s \rangle|\text{cats})) = -1$
- $\log(P(\text{likes}|\text{Mary})) = -10$
- $\log(P(\text{cats}|\text{likes})) = -100$

Assuming our test set is $W = \langle s \rangle \text{ Mary likes cats } \langle s \rangle$, what is the model's perplexity.

$\log PP(W) = (-1/5) * 113$

$\log PP(W) = (-1/4) * (-113)$

$\log PP(W) = -113$

$\log PP(W) = (-1/5) * (-113)$

1.7 词汇量匮乏时

分享

Outline

- Unknown words
- Update corpus with <UNK>
- Choosing vocabulary

known by building a vocabulary.

未知单词

分享

按 Esc 即可退出全屏模式

Out of vocabulary words

- Closed vs. Open vocabularies
- Unknown word = Out of vocabulary word (OOV)
- special tag <UNK> in corpus and in input

every unknown word with UNK.

Using <UNK> in corpus

- Create vocabulary V
- Replace any word in corpus and not in V by <UNK>
- Count the probabilities with <UNK> as with any other word

just with the addition of UNK.

deeplearning.ai

用 UNK 更新语料库

用最小频率筛选词汇表和 UNK

Example

Corpus

<s> Lyn drinks chocolate </s>
<s> John drinks tea </s>
<s> Lyn eats chocolate </s>



Corpus

<s> Lyn drinks chocolate </s>
<s> <UNK> drinks <UNK> </s>
<s> Lyn <UNK> chocolate </s>

Min frequency f=2

Vocabulary

Lyn, drinks, chocolate

Input query

<s> Adam drinks chocolate </s>



<s> <UNK> drinks chocolate </s>

character by character

deeplearning.ai

using deep learning,

选择词汇

最小频数只是一个启发式的想法，或者可以自定义选取指定频数到最大频数之间的词汇。

要考虑的一件事是 UNK 对困惑的影响。它会降低还是提高它？实际上，它通常更低。所以看起来像您的语言模型越来越好。不过也需要谨慎调节这种筛选规则，不然很多有意义的句子看起来就像一串 UNK 的组合，效果反而不好。

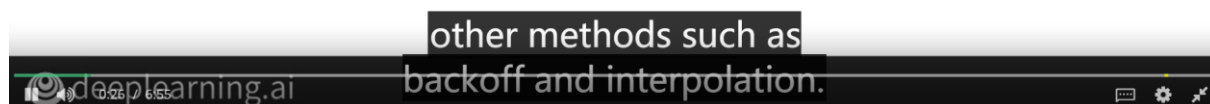
1.8 平滑处理

单词频率可能会倾斜，需要进行平滑补救。

分享

Outline

- Missing N-grams in corpus
- Smoothing
- Backoff and interpolation



- 语料库信息缺失

分享

Example

Corpus

<s> Lyn drinks chocolate </s>
<s> John drinks tea </s>
<s> Lyn eats chocolate </s>



Corpus

<s> Lyn drinks chocolate </s>
<s> <UNK> drinks <UNK> </s>
<s> Lyn <UNK> chocolate </s>

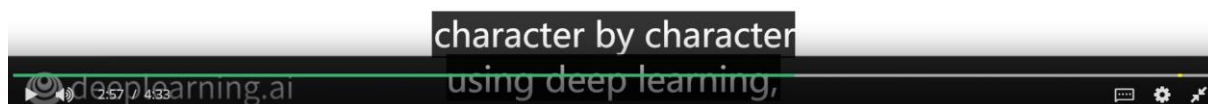
Min frequency $f=2$

Vocabulary

Lyn, drinks, chocolate

Input query

<s> Adam drinks chocolate </s>
↓
<s> <UNK> drinks chocolate </s>



这个例子中，John eats 这个二元组会消失，因为都被标记成了 UNK，所以造成了信息缺失。

- 平滑处理

Smoothing

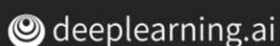
- Add-one smoothing (Laplacian smoothing)

$$P(w_n|w_{n-1}) = \frac{C(w_{n-1}, w_n) + 1}{\sum_{w \in V} (C(w_{n-1}, w) + 1)} = \frac{C(w_{n-1}, w_n) + 1}{C(w_{n-1}) + V}$$

- Add-k smoothing

$$P(w_n|w_{n-1}) = \frac{C(w_{n-1}, w_n) + k}{\sum_{w \in V} (C(w_{n-1}, w) + k)} = \frac{C(w_{n-1}, w_n) + k}{C(w_{n-1}) + k * V}$$

higher order n-gram
probabilities as well,



- 回退和插值

按 Esc 即可退出全屏模式

Backoff

- If N-gram missing => use (N-1)-gram, ...
 - Probability discounting e.g. Katz backoff
 - "Stupid" backoff

Corpus

<s> Lyn drinks chocolate </s>

<s> John drinks tea </s>

<s> Lyn eats chocolate </s>

$P(chocolate|John\ drinks) = ?$



$0.4 \times P(chocolate|drinks)$

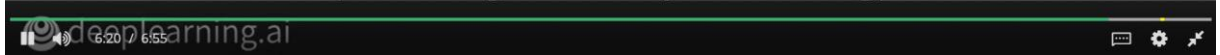
0.4 would be used instead.

Interpolation

$$\hat{P}(\text{chocolate}|\text{John drinks}) = 0.7 \times P(\text{chocolate}|\text{John drinks}) \\ + 0.2 \times P(\text{chocolate}|\text{drinks}) + 0.1 \times P(\text{chocolate})$$

$$\hat{P}(w_n|w_{n-2} w_{n-1}) = \lambda_1 \times P(w_n|w_{n-2} w_{n-1}) \\ + \lambda_2 \times P(w_n|w_{n-1}) + \lambda_3 \times P(w_n) \quad \sum_i \lambda_i = 1$$

You can get them by maximizing



Corpus: “I am happy I am learning”

In the context of our corpus, what is the estimated probability of word “can” following the word “I” using the bigram model and add-k-smoothing where k=3.

$$P(\text{can}|\text{I}) = 0$$

$$\mathbf{P(\text{can}|\text{I}) = 3/(2+3*4)}$$

$$P(\text{can}|\text{I}) = 3/(3*4)$$

$$P(\text{can}|\text{I}) = 1$$

2. 词嵌入

2.1 大纲介绍

Learning objectives

Prerequisite: neural networks

- Identify the key concepts of word representations
- Generate word embeddings
- Prepare text for machine learning
- Implement the continuous bag-of-words model

of the deep learning
specialization by deeplearning.ai

2.2 独热向量

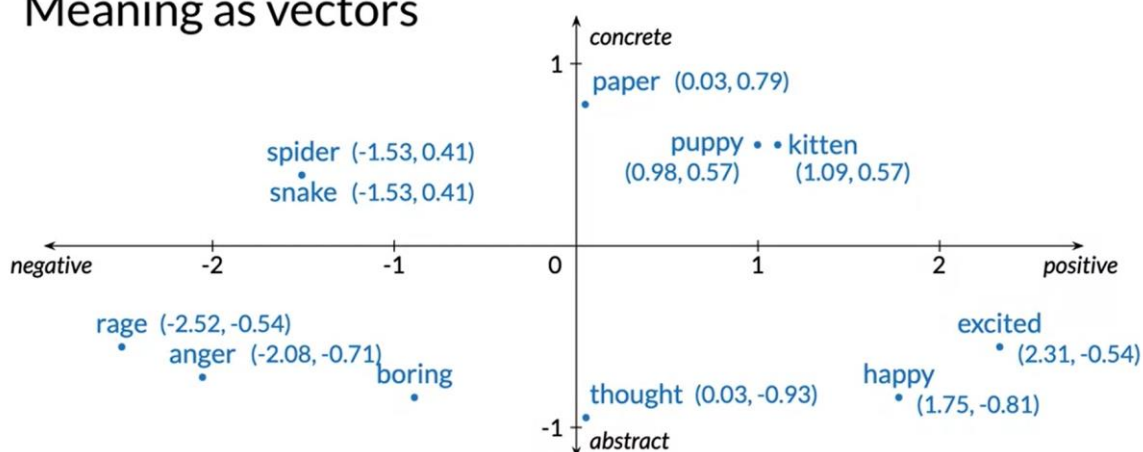
将单词根据语料库的大小，编辑成独热向量的方式是最基本的词嵌入方法。这种方法有两个大的缺点：

- 语料库很大的时候，每个单词都需要大量的内存，并且进行数据处理时耗时更长，存储数据的矩阵非常稀疏。
- 每个向量完全独立，无法衡量两个单词之间的相似性。

2.3 词嵌入

什么叫词嵌入？word embedding？即带有含义的词向量，独热向量只能称为词向量。

Meaning as vectors



like snake and spider for example.



deeplearning.ai



Summary

- Words as integers
- Words as vectors
 - One-hot vectors
 - Word embedding vectors
- Benefits of word embeddings for NLP

Up next, you will create word embeddings.

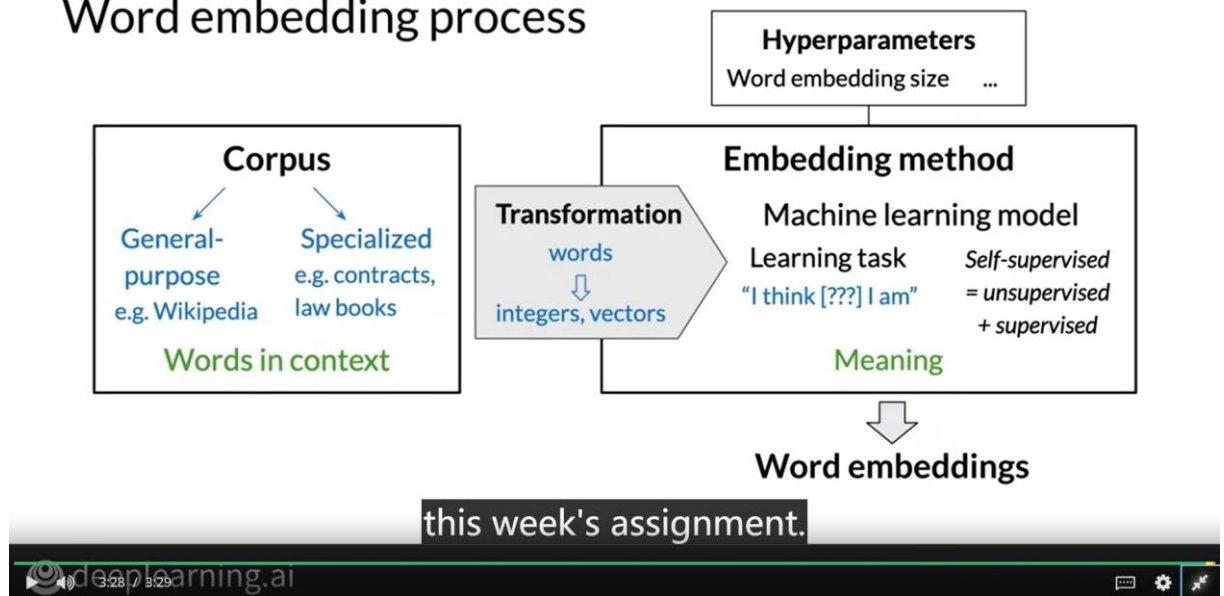


deeplearning.ai

如何创造词嵌入?

语料库 和 嵌入方法, 是两个重要的决定因素。

Word embedding process



通俗地讲，这项任务是自我监督的。从输入数据（语料库）未标记的意义上讲，这两者都是无监督的。在某种意义上，数据本身提供了通常构成标签的必要上下文。因此，语料库是一个自包含的数据集，它既包含训练数据，也包含能够监督任务的数据。

2.4 词嵌入方法模型介绍

Basic word embedding methods

- word2vec (Google, 2013)
 - Continuous bag-of-words (CBOW)
 - Continuous skip-gram / Skip-gram with negative sampling (SGNS)
- Global Vectors (GloVe) (Stanford, 2014)
- fastText (Facebook, 2016)
 - Supports out-of-vocabulary (OOV) words

make vector representations
of phrases and sentences.

deeplearning.ai

Word2vec 由 Google 开发，它最初普及了机器学习的使用，用来生成单词嵌入。Word2vec，使用浅层神经网络来学习单词嵌入。它提出了两个模型架构，连续的词袋模型，

这是一个简单但有效的方法，您将在本周实现。该模型的目标是学习在给定周围单词的情况下预测缺失的单词。连续跳转，又称为带负采样的跳转，与词袋模型的方法相反。该模型学习预测，给定输入词周围的单词。

还有 Facebook 的 fastText，它基于 skip gram 模型，通过将单词表示为 n 个字符来考虑单词的结构。这使得模型能够支持以前看不见的单词，也就是外部词汇表单词，方法是根据它们所组成的字符序列和最初训练它的相应序列推断出它们的嵌入。例如，它将为 kitty 和 kitten 创建类似的嵌入，即使它以前从未见过 kitty 这个词。因为 kitty 和 kitten 是由相似的字符序列组成的。fastText 的另一个好处是，单词嵌入向量可以被平均在一起，从而对短语和句子进行向量表示。

更高级的词嵌入模型：

Advanced word embedding methods

Deep learning, contextual embeddings

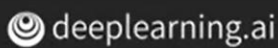
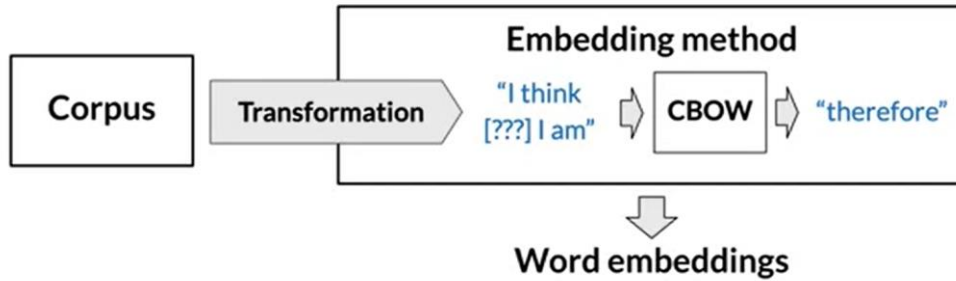
- BERT (Google, 2018)
 - ELMo (Allen Institute for AI, 2018)
 - GPT-2 (OpenAI, 2018)
- } Tunable pre-trained models available

your own corpus to
generate high-quality,
deeplearning.ai

2.5 连续词袋模型 (CBOW)

continue bag of words

Continuous bag-of-words word embedding process

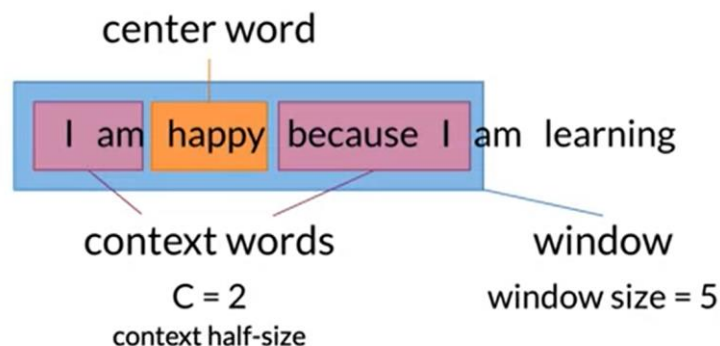


a missing word based on
the surrounding words.

要创建单词嵌入，需要执行学习任务的语料库和机器学习模型。学习任务的一个副产品是一组字嵌入。您还需要一种方法将语料库转换为适合机器学习模型的代表。在连续袋词模型的情况下，任务的目标是基于周围词预测缺失词。其理由是，如果两个独特的词在不同句子中使用时常被类似的词所包围，那么这两个词的意义往往是相关的。另一种说法是，它们在语义上是相关的。

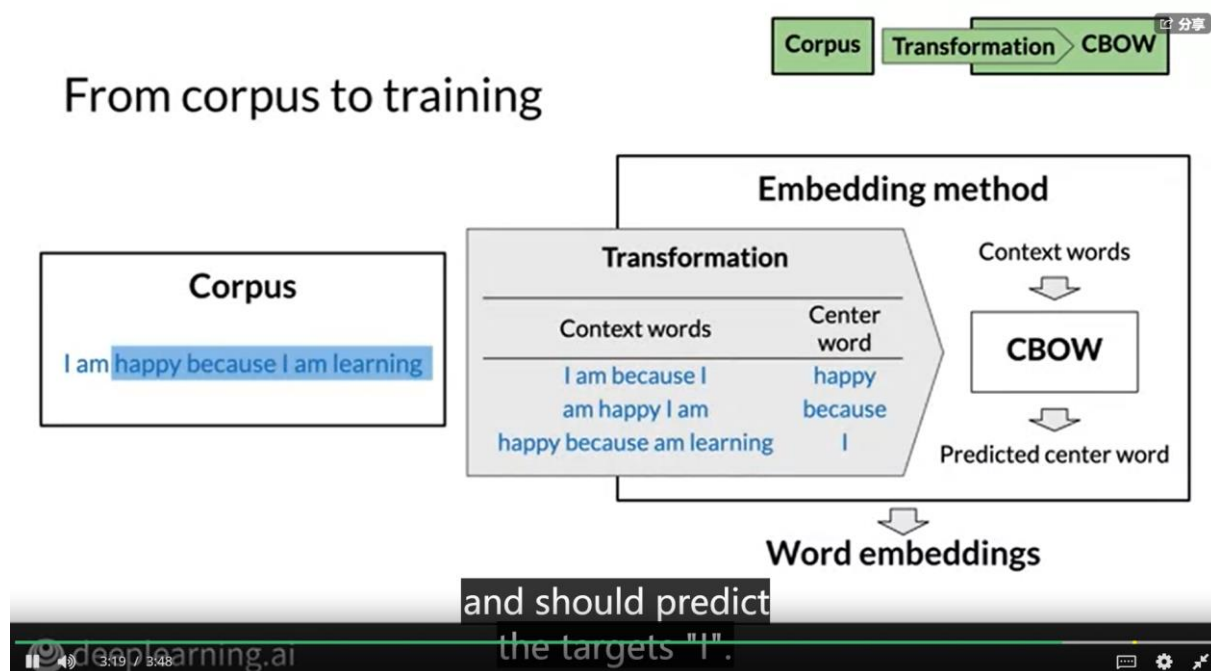
下图的参数指的是 文本半长 C ，窗口长度为 $2 * C + 1$

Creating a training example

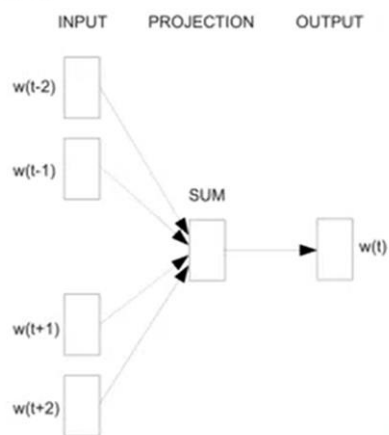


contexts words after
which equals to five.

词嵌入流程：



CBOW in a nutshell



Source: Mikolov, T., Chen, K., Corrado, G.S., & Dean, J. (2013). [Efficient Estimation of Word Representations in Vector Space](#)

you'll focus on preparing

deeplearning.ai

2.6 清理语料库、令牌化（标记化过程）

- 单词不区分大小写
- 处理标点符号
- 处理数字（若意义不大，可以删除；若某一些数字有唯一的意义，进行合适的替换效

果会更好，例如电话区号的替换)

- 特殊符号的处理，\$ 货币符号，分段符号。(通常删掉即可)
- 现在推特常有特殊表情符号和标签

Cleaning and tokenization matters

- Letter case "The" == "the" == "THE" → lowercase / upper case
- Punctuation , ! . ? → . " ' « » ' " → ∅ ... !! ??? → .
- Numbers 1 2 3 5 8 → ∅ 3.14159 90210 → as is / <NUMBER>
- Special characters ∇ \$ € § ¶ ** → ∅
- Special words 😊 #nlp → :happy: #nlp

hashtag should be considered
as an individual word.

deeplearning.ai

下面将介绍一个简单的 token 过程的例子：

这个是原文本

分享

Example in Python: corpus

Who ❤️ "word embeddings" in 2020? I do!!!

emoji punctuation number

punctuation marks,
and the number.

deeplearning.ai

引用文本处理的库函数

Example in Python: libraries

```
# pip install nltk
# pip install emoji

import nltk
from nltk.tokenize import word_tokenize
import emoji

nltk.download('punkt') # download pre-trained Punkt tokenizer for English
```



deeplearning.ai

I'm using the
popular NLTK library

Example in Python: code

```
corpus = 'Who ❤️ "word embeddings" in 2020? I do!!!'

data = re.sub(r'[,!?;-]+', '.', corpus)
data = nltk.word_tokenize(data) # tokenize string to words

→ ['Who', '❤️', '``', 'word', 'embeddings', '"', 'in', '2020', '.', 'I', 'do', '.']
```



deeplearning.ai

including the quotation marks.

Example in Python: code

```
corpus = 'Who ❤️ "word embeddings" in 2020? I do!!!'

data = re.sub(r'[!?;-]+', '.', corpus)
data = nltk.word_tokenize(data) # tokenize string to words
data = [ ch.lower() for ch in data
        if ch.isalpha()
        or ch == '.'
        or emoji.get_emoji_regexp().search(ch)
        ]
```

→ ['who', '❤️', 'word', 'embeddings', 'in', '.', 'i', 'do', '.']

You can now use this
array to extract

python 中的单词滑动窗口:

Sliding window of words in Python

```
def get_windows(words, C):
    i = 0
    while i < len(words) - C:
        center_word = words[i]
        context_words = words[(i - C):i] + words[(i+1):(i+C+1)]
        yield context_words, center_word
        i += 1
```

I	am	happy	because	I	am	learning
0	1	2	3	4	5	6

the number of words in the array

deeplearning.ai

使用了 yield, 将函数做成一个生成器。多次迭代使用即可。

Sliding window of words in Python

```
for x, y in get_windows(  
    ['i', 'am', 'happy', 'because', 'i', 'am', 'learning'],  
    2  
):  
    print(f'{x}\t{y}')
```

→ ['I', 'am', 'because', 'I'] happy
 ['am', 'happy', 'I', 'am'] because
 ['happy', 'because', 'am', 'learning'] I

and the contexts
have size of two,

deeplearning.ai

2.7 将单词转化为向量

下面是 C=2 的一个向量化示例：为方便理解，使用的是独热编码。

Transforming center words into vectors

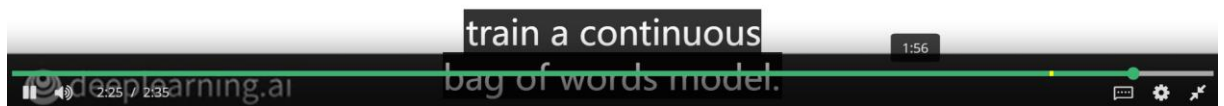
Corpus	I am happy because I am learning				
Vocabulary	am, because, happy, I, learning				
One-hot vector	am	because	happy	I	learning
	am	because	happy	I	learning
	1	0	0	0	0
	0	1	0	0	0
	0	0	1	0	0
	0	0	0	1	0
	0	0	0	0	1

This is how you'll input the
center words into the model.

deeplearning.ai

Final prepared training set

Context words	Context words vector	Center word	Center word vector
<i>I am because I</i>	[0.25; 0.25; 0; 0.5; 0]	<i>happy</i>	[0; 0; 1; 0; 0]
<i>am happy I am</i>	[0.5; 0; 0.25; 0.25; 0]	<i>because</i>	[0; 1; 0; 0; 0]
<i>happy because am learning</i>	[0.25; 0.25; 0.25; 0; 0.25]	<i>I</i>	[0; 0; 0; 1; 0]



2.8 CBOW 的架构和原理推导

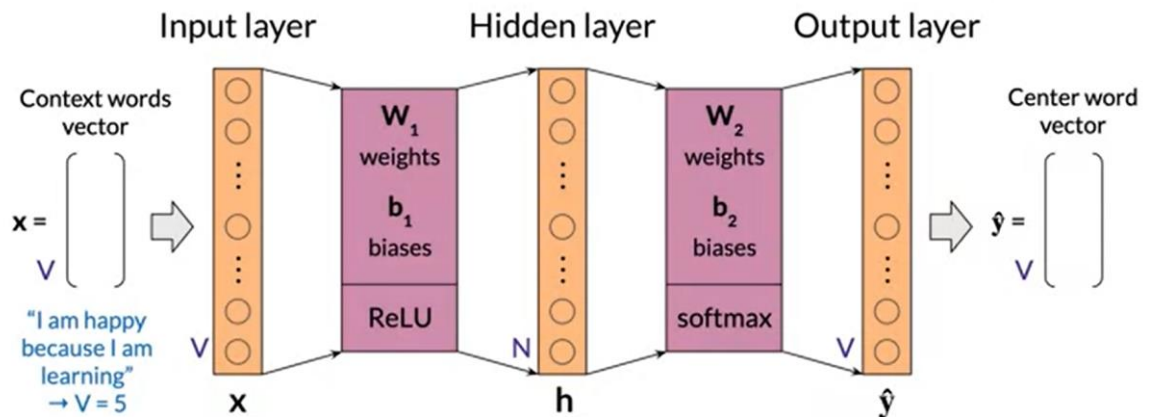
CBOW 的体系架构：

输入和输出层的向量大小和语料库的大小 V 有关。

隐藏层和嵌入单词 N 的大小有关，即嵌入单词的维数（积极消极，形象抽象等维度的向量表示）。

Architecture of the CBOW model

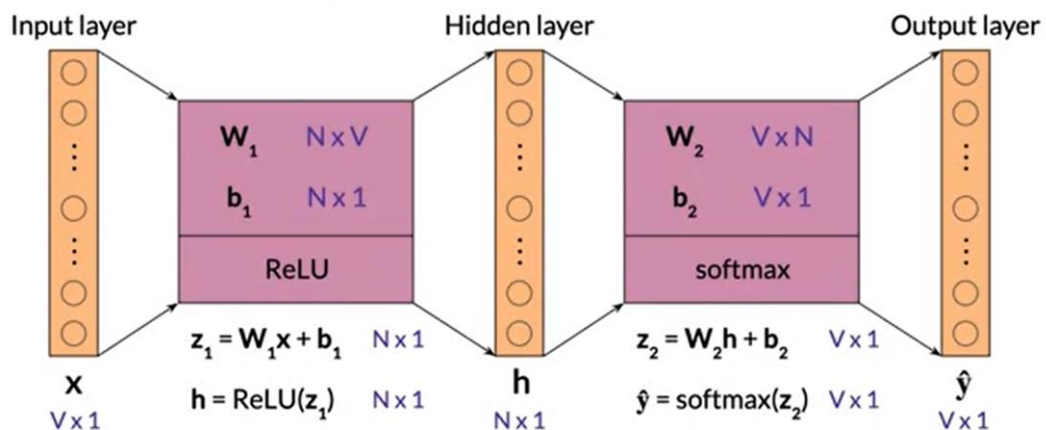
Hyperparameters
N: Word embedding size ...



There you have it,
the architecture

CBOW 的体系结构：向量、矩阵的维数

Dimensions (single input)



for the hidden layer h and
add the bias vector b_2 ,

	意义	维数
x	输入层向量	$V \times 1$
W_1	输入层到隐藏层的权重矩阵	$n \times V$
b_1	输入层到隐藏层的偏差项	$n \times 1$
z_1	$W_1 \times x + b_1$	$n \times 1$
h	$\text{Relu}(z_1)$	$n \times 1$
W_2	隐藏层到输出层的权重矩阵	$V \times n$
b_2	隐藏层到输出层的偏差项	$V \times 1$
y	$\text{softmax}(W_2 \times h + b_2)$	$V \times 1$

Dimensions (single input)

Column vectors

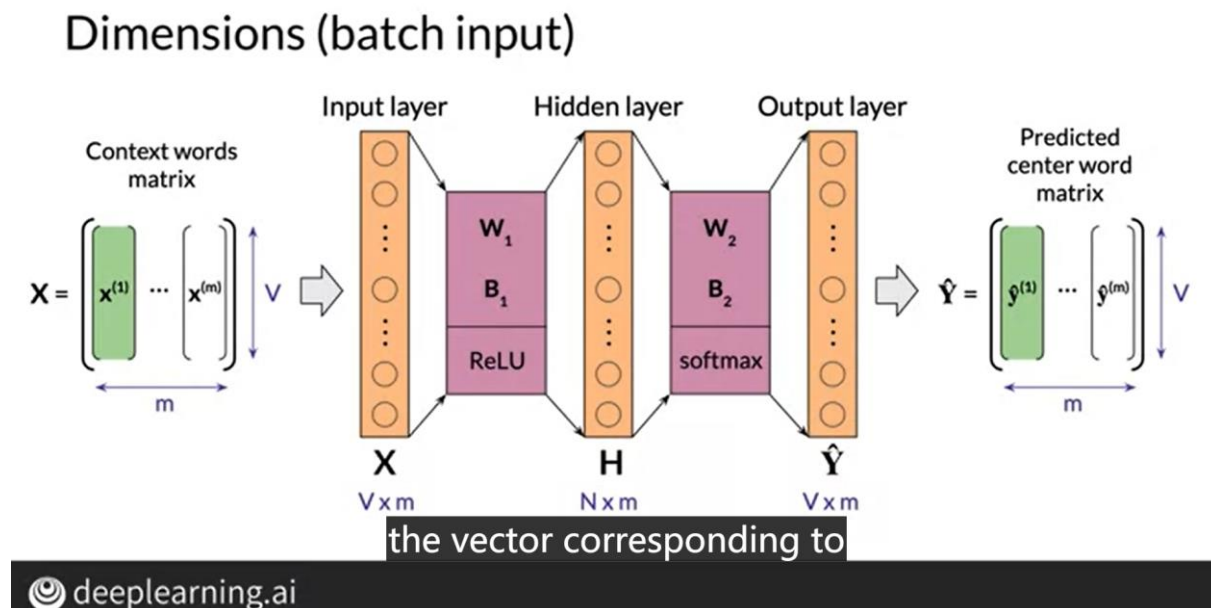
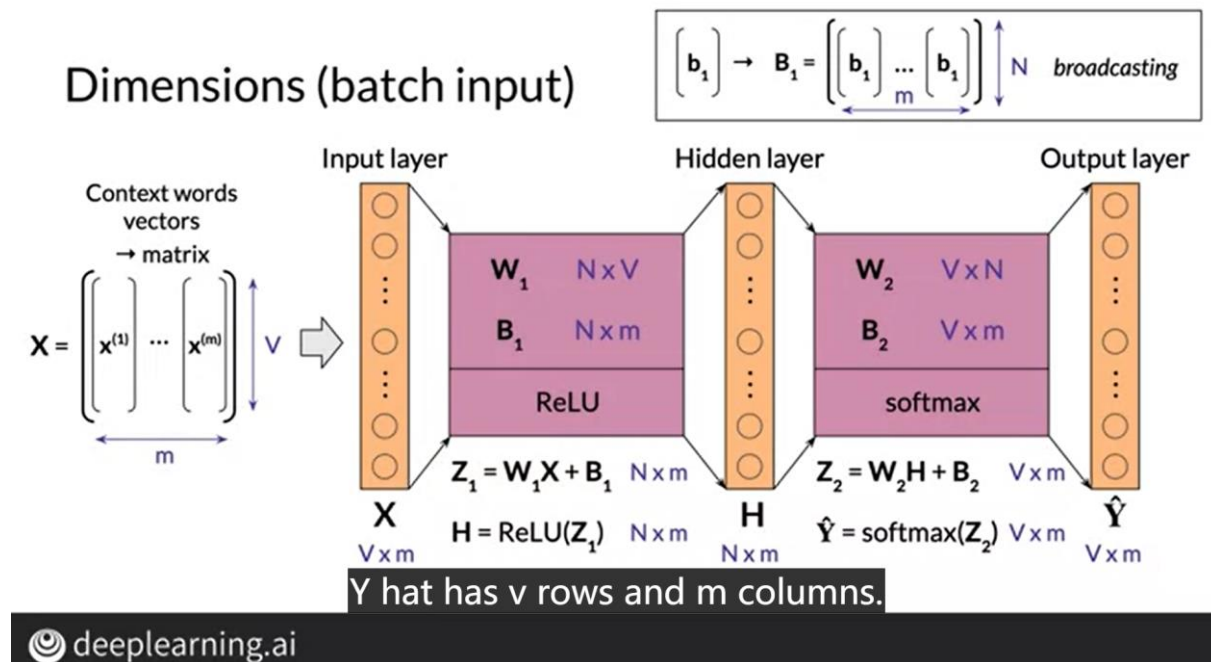
$$z_1 = W_1 x + b_1 \quad \begin{matrix} z_1 = \begin{bmatrix} \\ \\ \end{bmatrix}_{N \times 1} & W_1 = \begin{bmatrix} & & \\ & & \\ N \times V & & \end{bmatrix} & x = \begin{bmatrix} \\ \\ \end{bmatrix}_{V \times 1} & b_1 = \begin{bmatrix} \\ \\ \end{bmatrix}_{N \times 1} \end{matrix}$$

Row vectors

$$z_1 = x W_1^T + b_1 \quad \begin{matrix} b_1 = \begin{bmatrix} 1 \times N \end{bmatrix} & W_1 = \begin{bmatrix} & & \\ & & \\ N \times V & & \end{bmatrix} & b_1 = \begin{bmatrix} 1 \times N \end{bmatrix} \\ x = \begin{bmatrix} 1 \times V \end{bmatrix} & & \end{matrix}$$

Next, I'll show you how to
use the neural network with

批量化的 CBOW 的体系结构：矩阵的维数



If you are designing a neural network for a CBOW model that will be trained on a corpus of 8000 words, and if you want it to learn 400-dimensional word embedding vectors, what should be the size of W_1 , the weighting matrix between the input layer and hidden layer, if it is fed training examples in batches of 16 examples represented by a 8000 row by 16 column matrix?

Hint: if X is the input matrix, H the matrix for the hidden layer, and $B1$ the bias matrix, then $H = \text{ReLU}(W1X + B1)$.

16 rows by 8000 columns.

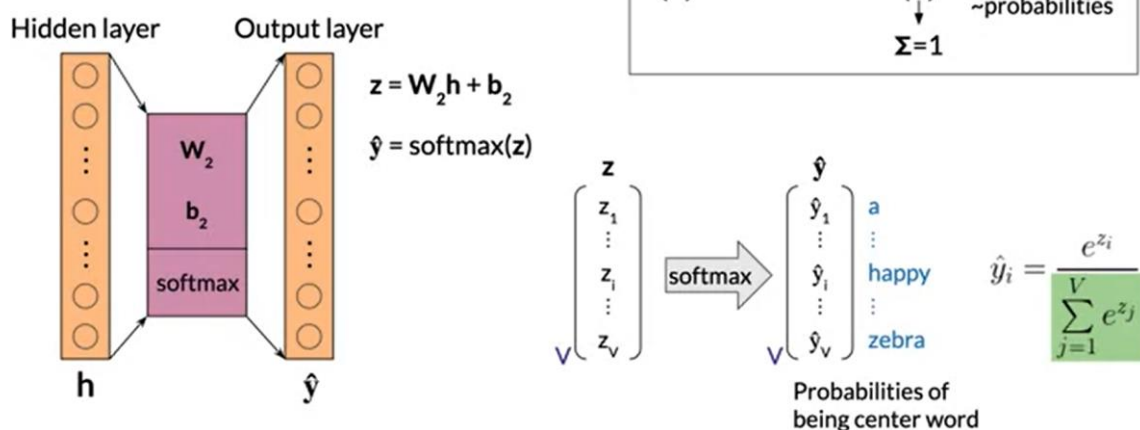
400 rows by 8000 columns.

8000 rows by 16 columns

400 rows by 16 columns.

激活函数的梳理介绍

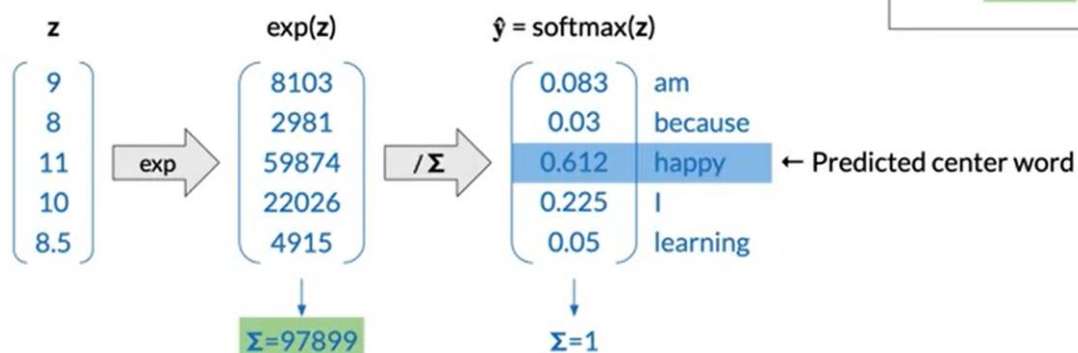
Softmax



the divisor normalizes the vector so that the rows sum up to 1.

deeplearning.ai

Softmax: example



$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^v e^{z_j}}$$

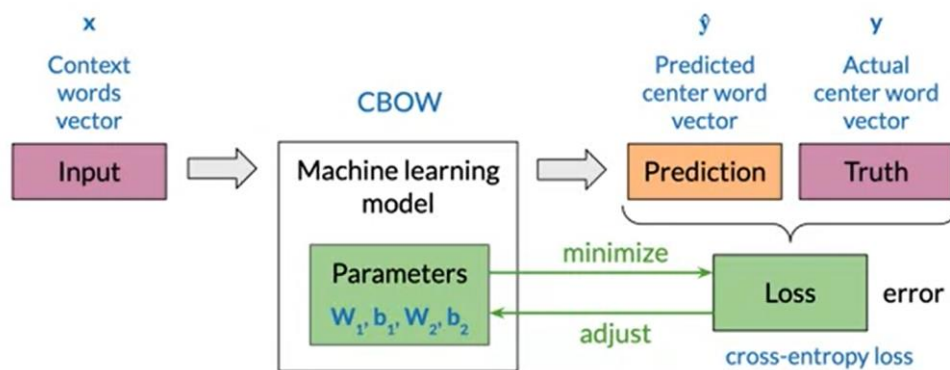
You'll get to implement both relu and

deeplearning.ai

2.9 算法推导

2.9.1 代价函数

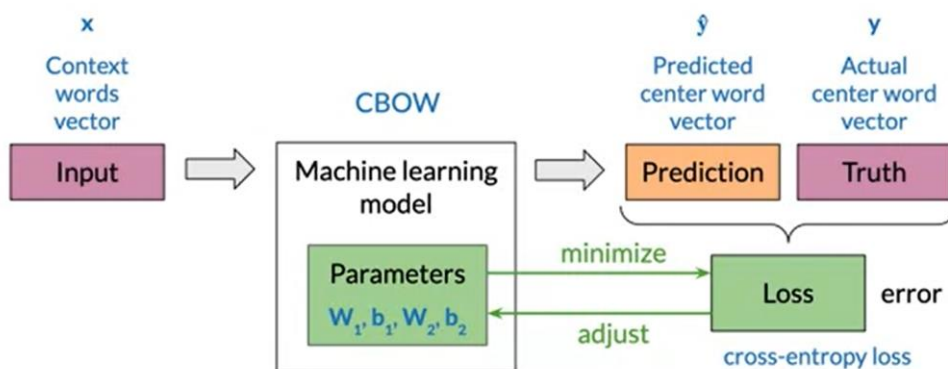
Loss



a simple form of the
cross-entropy loss function,

预测正确时:

Loss



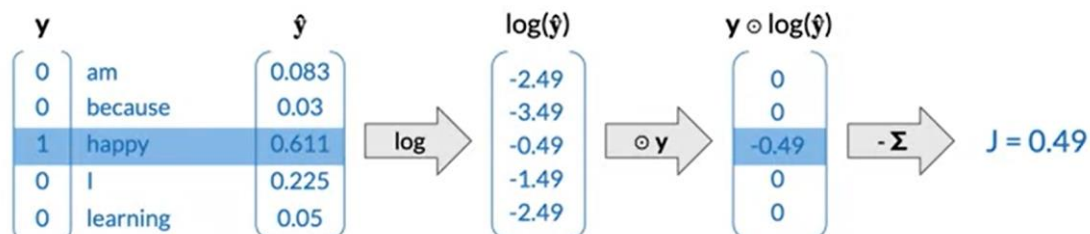
a simple form of the
cross-entropy loss function,

Cross-entropy loss

$$J = - \sum_{k=1}^V y_k \log \hat{y}_k$$

Actual	Predicted
$\mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_v \end{bmatrix}$	$\hat{\mathbf{y}} = \begin{bmatrix} \hat{y}_1 \\ \vdots \\ \hat{y}_v \end{bmatrix}$

I am happy because I am learning



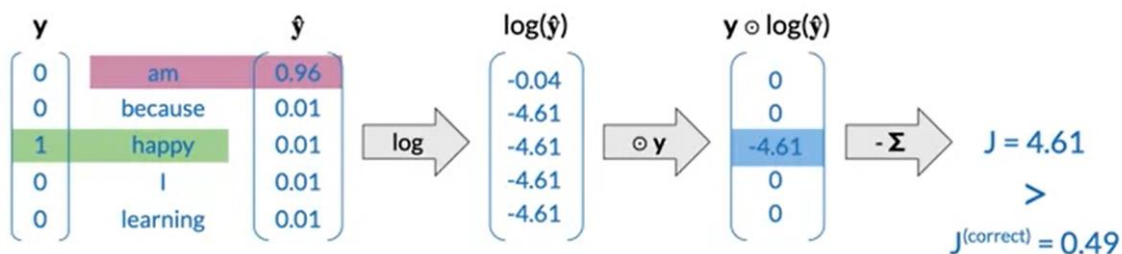
this value and the loss is
minus the sum, so 0.49.



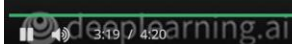
预测错误时：

Cross-entropy loss

$$J = - \sum_{k=1}^V y_k \log \hat{y}_k$$

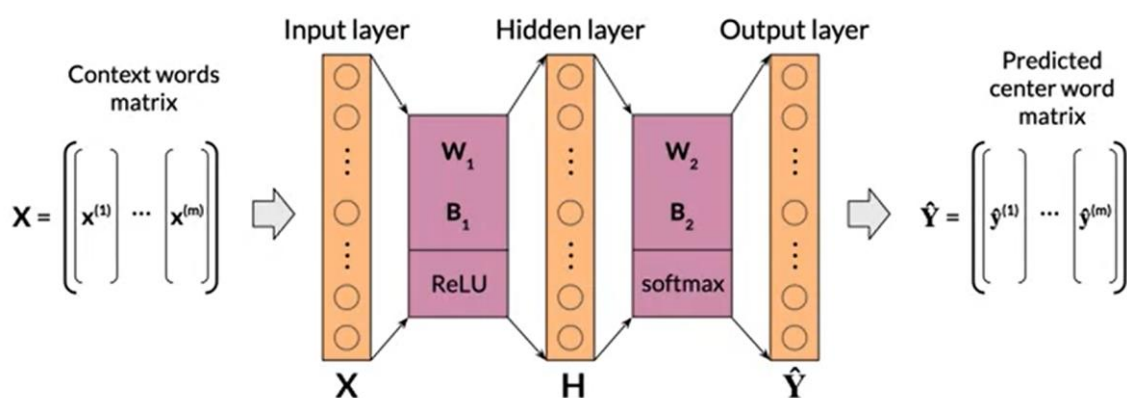


So you can see that the loss is



2.9.2 正向传播

Forward propagation



corresponding to the m input vectors

deeplearning.ai representing each of training examples.

Cost

Cost: mean of losses

$$J = - \sum_{k=1}^V y_k \log \hat{y}_k$$

$$J_{batch} = -\frac{1}{m} \sum_{i=1}^m \sum_{j=1}^V y_j^{(i)} \log \hat{y}_j^{(i)}$$

$$J_{batch} = -\frac{1}{m} \sum_{i=1}^m J^{(i)}$$

Predicted center word matrix

$$\hat{\mathbf{Y}} = \begin{bmatrix} \begin{bmatrix} \hat{y}^{(1)} \end{bmatrix} & \dots & \begin{bmatrix} \hat{y}^{(m)} \end{bmatrix} \end{bmatrix}$$

Actual center word matrix

$$\mathbf{Y} = \begin{bmatrix} \begin{bmatrix} y^{(1)} \end{bmatrix} & \dots & \begin{bmatrix} y^{(m)} \end{bmatrix} \end{bmatrix}$$

improve its predictions.

deeplearning.ai

2.9.3 反向传播和梯度下降

分享

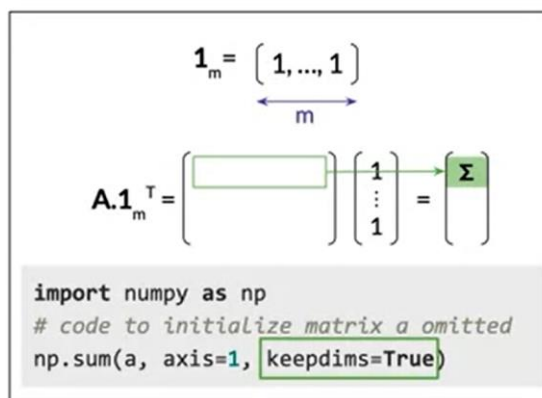
Backpropagation

$$\frac{\partial J_{batch}}{\partial \mathbf{W}_1} = \frac{1}{m} \text{ReLU}(\mathbf{W}_2^\top (\hat{\mathbf{Y}} - \mathbf{Y})) \mathbf{X}^\top$$

$$\frac{\partial J_{batch}}{\partial \mathbf{W}_2} = \frac{1}{m} (\hat{\mathbf{Y}} - \mathbf{Y}) \mathbf{H}^\top$$

$$\frac{\partial J_{batch}}{\partial \mathbf{b}_1} = \frac{1}{m} \text{ReLU}(\mathbf{W}_2^\top (\hat{\mathbf{Y}} - \mathbf{Y})) \mathbf{1}_m^\top$$

$$\frac{\partial J_{batch}}{\partial \mathbf{b}_2} = \frac{1}{m} (\hat{\mathbf{Y}} - \mathbf{Y}) \mathbf{1}_m^\top$$



Now for the purposes of this course,

you don't have to understand how these

小问题：下面的参数更新之后的维数是多少呢？（看上图可得）

Gradient descent

Hyperparameter: learning rate α

$$\mathbf{W}_1 := \mathbf{W}_1 - \alpha \frac{\partial J_{batch}}{\partial \mathbf{W}_1}$$

$$\mathbf{W}_2 := \mathbf{W}_2 - \alpha \frac{\partial J_{batch}}{\partial \mathbf{W}_2}$$

$$\mathbf{b}_1 := \mathbf{b}_1 - \alpha \frac{\partial J_{batch}}{\partial \mathbf{b}_1}$$

$$\mathbf{b}_2 := \mathbf{b}_2 - \alpha \frac{\partial J_{batch}}{\partial \mathbf{b}_2}$$

In the next video, you'll learn how to

extract word embedding vectors from

梯度下降的原理（仅针对单个输入）推导看下图，纯手推（不完善）：

单个例子:

$$\begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_v \end{pmatrix}_{v \times 1} \xrightarrow{W_1 x + b_1} \begin{pmatrix} z_{11} \\ z_{12} \\ \vdots \\ z_{1v} \end{pmatrix}_{n \times 1} \xrightarrow{\text{RELU}} \begin{pmatrix} h_1 \\ h_2 \\ \vdots \\ h_n \end{pmatrix}_{n \times 1} \xrightarrow{W_2 h + b_2} \begin{pmatrix} z_{21} \\ z_{22} \\ \vdots \\ z_{2v} \end{pmatrix}_{v \times 1} \xrightarrow{\text{softmax}} \begin{pmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_v \end{pmatrix}_{v \times 1}$$

交叉熵损失函数:

$$J_{\text{cost}} = - \sum_{j=1}^v y_j \log \hat{y}_j$$

由于本例只针对一个输入计算误差逆传播算法和参数更新,

$$Y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_v \end{pmatrix}, \quad \hat{Y} = \begin{pmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_v \end{pmatrix} = \begin{pmatrix} \frac{e^{z_{11}}}{\sum_{k=1}^v e^{z_{1k}}} \\ \vdots \\ \frac{e^{z_{1v}}}{\sum_{k=1}^v e^{z_{1k}}} \end{pmatrix}$$

在 $j=1, \dots, v$ 中, 对 Y 而言, 仅有一个索引下的 y_j 值为 1, 其余均为 0. 故

$$J_{\text{cost}} = -y_j \log \hat{y}_j = -\log \hat{y}_j$$

$$\begin{cases} \frac{\partial J}{\partial \hat{y}_j} = -\frac{1}{\hat{y}_j} \end{cases}$$

这里要分情况讨论 $\hat{y}_j$ 对 z_i 的导数:

当 $i=j$ 时,

$$\frac{\partial \hat{y}_j}{\partial z_i} = \frac{\partial}{\partial z_j} \left(\frac{e^{z_j}}{\sum_k e^{z_k}} \right) = \frac{e^{z_j} (\sum_k e^{z_k}) - e^{z_j} \cdot e^{z_j}}{(\sum_k e^{z_k})^2} = \frac{e^{z_j}}{\sum_k e^{z_k}} - \frac{e^{z_j}}{\sum_k e^{z_k}} \cdot \frac{e^{z_j}}{e^{z_j}} = \hat{y}_j - \hat{y}_j \cdot \hat{y}_j = \hat{y}_j (1 - \hat{y}_j)$$

当 $i \neq j$ 时

$$\frac{\partial \hat{y}_j}{\partial z_i} = \frac{0 \cdot (\sum_k e^{z_k}) - e^{z_j} \cdot e^{z_i}}{(\sum_k e^{z_k})^2} = -\hat{y}_j \cdot \hat{y}_i$$

故 $\frac{\partial J}{\partial z_i} = \frac{\partial J}{\partial \hat{y}_j} \cdot \frac{\partial \hat{y}_j}{\partial z_i} = \begin{cases} \hat{y}_j - 1 & \text{当 } i=j \text{ 时} \\ \hat{y}_j - 0 & \text{当 } i \neq j \text{ 时} \end{cases} = \hat{y}_j - y_j$

即 $\frac{\partial J}{\partial z_i} = \hat{y}_i - y_i$

而 $\frac{\partial z_2}{\partial W_2} = h^T$ (线性代数)

故 $\frac{\partial J}{\partial W_2} = \frac{\partial J}{\partial z_2} \cdot \frac{\partial z_2}{\partial W_2} = (\hat{Y} - Y) \cdot h^T$

$\frac{\partial J}{\partial b_2} = \frac{\partial J}{\partial z_2} \cdot \frac{\partial z_2}{\partial b_2} = (\hat{Y} - Y) \cdot 1$

$\frac{\partial J}{\partial W_1} = \frac{\partial J}{\partial z_1} \cdot \frac{\partial z_1}{\partial W_1} = \text{RELU}[W_2^T (\hat{Y} - Y)] \cdot x^T$

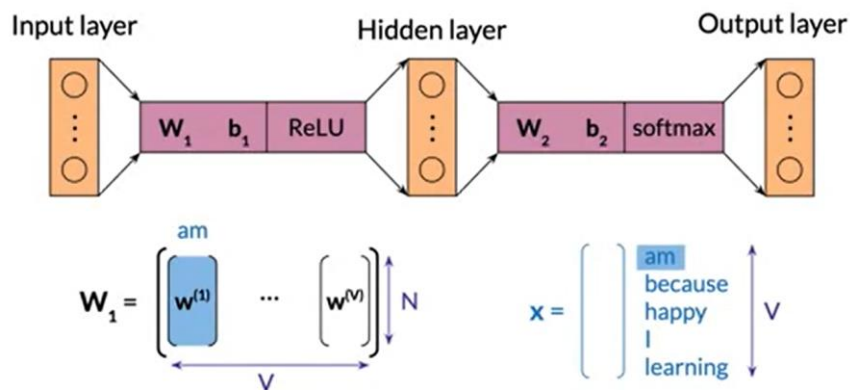
$\frac{\partial J}{\partial b_1} = \text{RELU}[W_2^T (\hat{Y} - Y)]$

对于批量输入的误差逆传播算法, 其维数稍有不同。由上图可推广, 懒得写啦!

2.10 提取词嵌入向量

词嵌入是预测模型的副产品, 接下来我们要学习如何提取词嵌入向量。

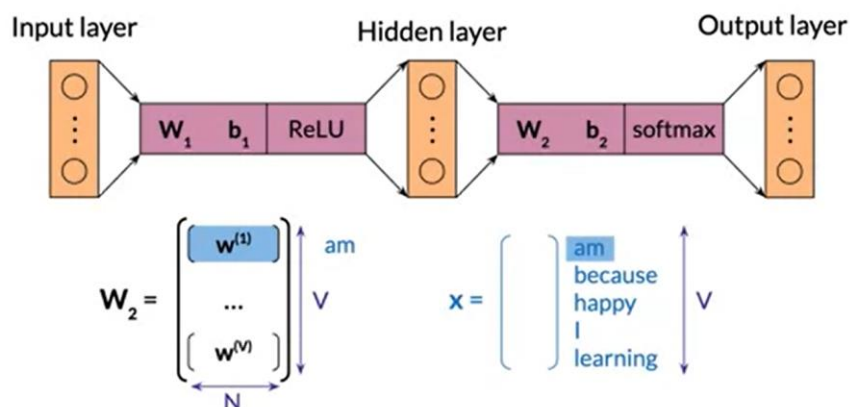
Extracting word embedding vectors: option 1



the word embedding

column vector for m,

Extracting word embedding vectors: option 2

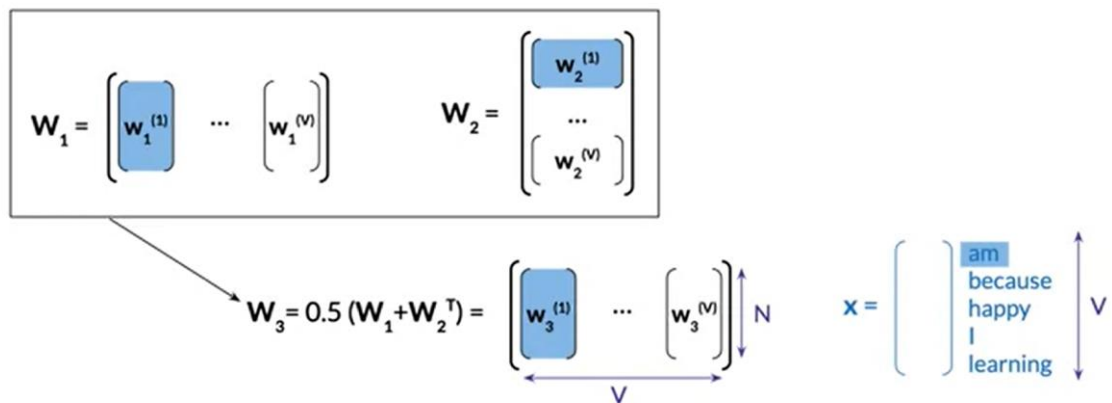


embedding row vector for m,

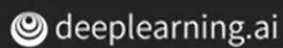
deeplearning.ai

W_1 的列可以看做对应单词的词嵌入， W_2 的行向量可以看做对应单词的词嵌入。因此有了下面的启发式想法：

Extracting word embedding vectors: option 3



the word embedding for m



做一个题目试试本节的知识点掌握了没。

试题

The following weighting matrix W_1 has been learned after training a CBOW model. You are also given word-to-row mapping for the input column vectors.

$$W_1 = \begin{bmatrix} 1.76 & -0.21 & 4.56 & -2.39 & 2.11 & -2.98 \\ 1.64 & -2.5 & -2.94 & -1.3 & 0.09 & 1.81 \\ -1.9 & -1.18 & 2.61 & -1.7 & -4.36 & -4.54 \\ -3.65 & -0.3 & -1.16 & 1.75 & -4.05 & -3.38 \end{bmatrix} \quad x = \begin{bmatrix} \text{all} \\ \text{one} \\ \text{ring} \\ \text{rule} \\ \text{them} \\ \text{to} \end{bmatrix}$$

Which of the following is a possible word embedding vector that you can extract using W_1 for the word "ring"?

- ☒ [4.56; -2.94; 2.61; -1.16]
- ☐ [-2.39; -1.3; -1.7; 1.75]
- ☐ [-1.9; -1.18; 2.61; -1.7; -4.36; -4.54]
- ☐ [0; 0; 1; 0; 0; 0]

跳过

继续

2.11 词嵌入的评估

2.11.1 内部评估

- 测试语义类比，比如：法国对巴黎，好比中国对北京。或者是这样的：

seen is to 'saw' as been is to '?

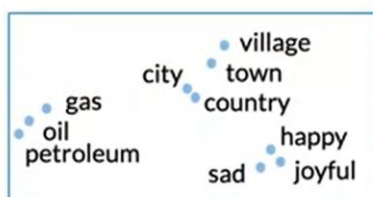
类推这种方法也是有缺点的，比如无法捕捉化学元素及符号之间的关系。

- 也可以使用聚类的方法：

Intrinsic evaluation

Test relationships between words

- Analogies
- Clustering
- Visualization



to evaluate the quality of word
embeddings, extrinsic evaluation.

2.11.2 外部评估

在外部任务中测试词嵌入的质量。例如，命名实体识别，词性标记等。确保词嵌入的实际意义是有用的，其主要缺点就是比内在评估更耗时，当性能很差的时候，那么性能指标（准确率，F1 分数）等不会提供任何帮助。

命名实体识别：将 钟南山 识别为一个人，将民主党识别为一个组织。

Extrinsic evaluation

Test word embeddings on external task

e.g. named entity recognition, parts-of-speech tagging

- + Evaluates actual usefulness of embeddings
- Time-consuming
- More difficult to troubleshoot

a comprehensive and very readable

 deeplearning.ai paper on evaluating word embeddings.